

Weekly Assignment 8: Divide-and-Conquer Algorithms

1. (15 points) We're about to analyse the complexity of a number of divide-and-conquer algorithms. Unfortunately, the algorithms themselves got lost. However, the recurrences measuring their worst case number of operations are still there. For each item, give the least $\mathcal{O}$ -class it belongs to. No proof or explanation needed.

(a)

$$T(n) = \begin{cases} 51 & \text{if } n = 1 \\ 5T(\frac{n}{5}) + 13 & \text{if } n = 5^k \text{ for some } k > 0 \end{cases}$$

(b)

$$T(n) = \begin{cases} 3 & \text{if } n = 1 \\ 3T(\frac{n}{3}) + n & \text{if } n = 3^k \text{ for some } k > 0 \end{cases}$$

(c)

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \\ 2T(n-1) & \text{if } n > 0 \end{cases}$$

2. (15 points) Let's consider a trivial example of a Divide and Conquer algorithm, which computes the sum of an array A of $n > 0$ numbers. If $n = 1$ and $A = [x]$ then return the result x . If $n > 0$ then split A in to equal halves, apply the procedure recursively, and return the sum of the results of both computations.

Analyse the complexity of this algorithm (assuming n is a power of 2), making use of a suitable recursive equation (and make this explicit in your analysis). Is this a useful application of D&C?

3. (15 points) Consider the following sorting algorithm which takes as input a list of integers A . During the algorithm, we keep a separate list B of integers that is already sorted, initially empty. The algorithm repeatedly selects the largest element of A , puts it at the end of B , and removes it from A . Then, it recursively sorts the rest. Once A is empty, we return the list B .

Show that this algorithm is in $\mathcal{O}(n^2)$, by writing down a recurrence for the number of comparisons in the worst case and solving it (through an educated guess, by repeated substitution, or using the Master theorem).

4. (25 points) Consider the following problem: given an integer array A of length $n > 1$, compute the following value:

$$\max\{A[j] - A[i] \mid 0 \leq i < j < n\}.$$

That is, we are looking for the maximal difference of two array elements, where the difference must be of the form $A[j] - A[i]$ with $i < j$, i.e., computed "from right to left". For instance:

- in the array $[62, -3, 5, 7, 2, 11]$ this value is 14 (the difference between -3 and 11);
- in the array $[1, 2, 3, 4, 5]$ this value is 4;
- in the array $[5, 4, 3, 2, 1]$ this value is -1.

Design an efficient divide-and-conquer algorithm for this problem. Discuss the correctness and complexity (for complexity, you may assume that the size n of the input array is a power of 2).

5. (30 points) Suppose we want to install software packages from a given set P of n packages. At most d packages in P (we don't know which ones) may have a *defect*: these packages can not be part of any successful installation. Assume there is a procedure `install( $Q$ )`, which attempts to install all the packages from a subset $Q \subseteq P$. This procedure has two possible outcomes: **success** when all packages of Q have been successfully installed, or **fail** in case the attempt was unsuccessful.
- (a) Assume an installation attempt `install( $Q$ )` is successful if and only if none of the packages in Q has a defect. Give an algorithm that uses at most $\mathcal{O}(d \log n)$ calls to `install` to successfully install a maximal subset of P . Briefly argue why your solution is correct and satisfies the requirements on the number of calls to `install`.
 - (b) Now assume that there may also be (known) *dependencies* between packages: we have a directed acyclic graph (DAG) $G = (P, K)$ such that for any pair $(p, q) \in K$ any installation that includes q but excludes p will fail. In this setting, an installation attempt `install( $Q$ )` is successful if and only if none of the packages in Q has a defect and moreover, for each dependency $(p, q) \in K$, $q \in Q$ implies $p \in Q$. Again give an algorithm that uses at most $\mathcal{O}(d \log n)$ calls to procedure `install` to successfully install a maximal subset of P . Briefly argue why your solution is correct and satisfies the requirements on the number of calls to `install`.